Restaurant Website Development Workflow

This document summarizes a professional workflow for building a restaurant website with QR-code based table ordering, suitable for a team project or freelance client.

1. Requirement Gathering

Meet the client to identify business goals, restaurant details, menu categories, table count, reservation needs, payment requirements, delivery/dine-in options, user roles (customer, waiter, kitchen, admin), branding, and hosting requirements. Produce a Software Requirement Specification (SRS).

2. Planning & Architecture

Create sitemap, user flow, database plan, API list, milestones, GitHub workflow, and sprint plan. Decide the technology stack and assign team responsibilities.

3. UI/UX Design

Design responsive wireframes and high-fidelity screens in Figma for Home, Menu, QR Order, Cart, Checkout, Admin Dashboard, Kitchen Dashboard, and Contact pages. Validate usability before coding.

4. Frontend Development

Develop using Next.js, TypeScript, Tailwind CSS, and reusable components. Implement routing, menu browsing, cart management, authentication (if needed), responsive layouts, and animations.

5. Backend Development

Build REST APIs with Express.js. Implement authentication, menu management, order creation, order status updates, table management, notifications, and validation.

6. Database

Use PostgreSQL with Prisma. Core tables: Restaurant, Categories, Menu Items, Tables, Orders, Order Items, Users, Payments (optional). Maintain relationships and indexes.

7. QR Ordering Workflow

Each dining table receives a unique QR code containing a URL such as /table/5. After scanning, customers see the menu for that table, add items to the cart, place the order, and receive order status updates. The kitchen and admin dashboards receive the order instantly using Socket.IO.

8. Admin Dashboard

Admins manage menu items, categories, tables, incoming orders, restaurant settings, analytics, and staff accounts.

9. Testing

Perform unit, integration, UI, responsiveness, browser compatibility, performance, and security testing. Verify QR scanning, order accuracy, and real-time updates.

10. Deployment & Maintenance

Deploy frontend (Vercel), backend (Railway/Render/Docker VPS), and PostgreSQL (Neon/Supabase). Configure backups, monitoring, logging, SSL, and environment variables.

Suggested Team Roles

Project Manager: planning and communication. UI/UX Designer: interfaces and design system. Frontend Developers: customer website and dashboard. Backend Developers: APIs, authentication, Socket.IO. Database Engineer: schema and optimization. QA Engineer: testing and

bug reporting.

Repository Strategy

Use a private repository while building a real client project. After completion, create a separate open-source starter template that removes branding, API keys, production configuration, and client data. This hybrid approach protects client assets while showcasing your technical skills.